RJH / ENGINEERING PUBLICATIONS

REFERENCE HANDBOOK • 2026

RJH

OFFLINE IoT DEVELOPMENT KIT COMPLETE GUIDE

From a blank laptop to a restored, editable Linux IoT platform.
Installation · Source code · Offline compilation · Integration · Troubleshooting

PREPARED BY RJH

EMQX Enterprise 6.3.1 / Kafka 4.2.2 / ThingsBoard CE 4.3.1.6

Zigbee2MQTT 2.14.2 / PostgreSQL 17 / Java bridge / Python BLE gateway

EDITION 2.0 • 13 NOVEMBER 2026 • OPEN SOURCE-KIT + LINUX DEVKIT

EDITION NOTES / RJH 2026

A complete, from-zero operating manual.

This edition upgrades the existing Source Kit installation handbook (33 pages, 30 October 2026) and Linux customization guide (27 pages, 10 November 2026). It incorporates the subsequent publication of 82 portable build chunks and supporting files on 13 November 2026, with beginner-safe installation instructions.

What this manual does: walks through pre-download planning, operating-system prerequisites, both releases, integrity validation, restore, Docker images, PostgreSQL, Kafka, each offline build toolchain, customizations, backup, security and a second-laptop test.

What it does not claim: a complete clean-machine restore or production certification, because those gates have not yet passed on the new laptop.

2	90	82
CONNECTED GITHUB REPOSITORIES	LINUX DEVKIT RELEASE ASSETS	VERIFIED PORTABLE DATA CHUNKS

Reading order

If you are starting from a clean computer, begin at Chapter 1 and work in order through the restore chapters. If the kit is already running, read the source editing and offline-build chapters. Do not skip the PostgreSQL decision and machine-isolation sections.

Evidence legend

Verified: original openSUSE compilation, local release checksums, public upload completion. **Candidate:** a plausible but not yet run clean-machine command or adaptation. **Blocked:** missing private backup, operating-system package repository, or hardware-dependent proof.

Copyright RJH — Technical guide compiled from published upstream repository sources and prior project manuals. Third-party product names, source code and licenses remain the property of their respective owners.

NAVIGATION / CLICKABLE CONTENTS

Contents

01. Purpose, edition, and what is actually verified **5**

02. One-page beginner roadmap **7**

03. Understand the architecture before installing anything **8**

04. Hardware, storage, and operating-system choices **10**

05. From zero: install openSUSE Leap 16.0 in VMware **11**

06. Prepare Linux tools and the offline boundary **12**

07. Know the two GitHub repositories and their releases **13**

08. Download everything while Internet is available **15**

09. Verify downloads, the USB transfer, and archive integrity **16**

10. Restore the original Source Kit baseline **17**

11. Understand the companion Linux DevKit contents **19**

12. Reassemble the 82 chunks and verify all seven archives **20**

13. Restore the portable Linux build workspace **21**

14. Load Docker images and prepare the baseline runtime **22**

15. PostgreSQL: distinguish new installation from backup restoration **23**

16. Start services safely and perform first smoke tests **25**

17. Directory map: know exactly what you can edit **27**

18. Git and source editing: a safe beginner workflow **28**

19. How strict offline build verification works **30**

20. EMQX Enterprise 6.3.1: edit and targeted recompilation **31**

21. Apache Kafka 4.2.2: an offline Java/Scala build **33**

22. ThingsBoard backend 4.3.1.6: Maven offline compile **35**

23. ThingsBoard Angular UI: language changes and production build **37**

24. Zigbee2MQTT 2.14.2: TypeScript and pnpm offline build **38**

25. The Java Kafka-to-ThingsBoard bridge **39**

26. The Python BLE-to-MQTT gateway **41**

27. Customize MQTT routing, Kafka topics, dashboards, and alerts **42**

28. Configure SMS providers and multilingual notifications **44**

29. Windows/WSL2 route: use only if you choose the legacy distribution path **45**

30. Daily operations, service backups, and safe rollback **46**

31. Troubleshooting: diagnosis before destructive changes **47**

32. GitHub release publishing and license review **49**

33. Full offline acceptance test: prove the new laptop really works **50**

34. Quick command reference for new maintainers **52**

35. Glossary, version pins, and source references **54**

36. Final installation and handover checklist **56**

Purpose, edition, and what is actually verified

This manual is a **from-zero installation and development handbook** for the two published `Developer20003` GitHub repositories. It combines the original seven-ZIP Source Kit, the native Linux DevKit, the newly uploaded portable build workspaces, and the practical customization procedures from the earlier manuals. It is written for a beginner who may be setting up a new laptop, creating an openSUSE virtual machine, or restoring the environment without Internet access.

> **PUBLISHED** Linux DevKit release: 90 assets on GitHub, including 82 data chunks and 6 supporting DevKit files plus 2 original starter assets. **NOT YET CERTIFIED** A different, freshly provisioned laptop has not yet passed the complete disconnected restore and modified-source rebuild acceptance test.

Two separate achievements must not be confused: (1) downloading and restoring prepared sources, compiler caches and Docker images; (2) proving that changed source files can be rebuilt and deployed entirely offline on a clean second machine. The first has been packaged and uploaded. The second remains an acceptance test.

Evidence level	Current meaning
Verified on the original openSUSE VM	Six offline compilation workflows passed using prepared workspaces/caches; source-kit services were running and selected message-path tests passed.
Verified release publication	Linux prerelease contains 90 uploaded assets totaling approximately **5.23 GB** (decimal); the 82 chunks and required manifests/scripts have been uploaded.
Not yet verified	Fresh-machine end-to-end offline restore; clean full-project EMQX rebuild; complete ThingsBoard distribution packaging; physical BLE/Zigbee device tests; production resilience.
Never included as public requirements	Private passwords, device tokens, running databases, customer records, VM disk images and a private PostgreSQL backup.

Use this document with the release files, not as a substitute for them. Command examples assume Bash on an x86_64 openSUSE Leap 16.0 machine unless a command is explicitly labeled Windows PowerShell or connected staging computer. The original Source Kit also supports a Windows/WSL/Docker path; the main path here is **native openSUSE Linux under VMware or on a laptop**.

Safety rule: Do not test destructive operations on an existing live kit. Never use `docker compose down -v`, reformat Kafka KRaft data, overwrite PostgreSQL data, or paste GitHub authentication tokens into reports or chat logs.

One-page beginner roadmap

A successful offline development environment starts online, then becomes offline. The offline computer cannot download the missing operating system, Docker engine, JDK, libraries, or compiler dependencies by itself. Do all acquisition on a connected staging computer before disconnecting the target.

Phase	Work location	Finished when...
0. Plan	Connected PC	You have a supported host, adequate disk, and selected Linux/VMware or Windows/WSL path.
1. Prepare OS	Target computer or VM, connected temporarily	openSUSE boots; storage, user account, Docker Engine, Compose, Git and Python are installed and tested.
2. Download	Connected staging PC	Both GitHub release asset sets and repository sources are stored together, with checksums.
3. Transfer	External SSD / trusted network share	Copy is complete; target has room for extraction, restore and a second working copy.
4. Restore baseline	Offline target	`iot-offline-development` contains sources, caches, runtime and configuration.
5. Reassemble DevKit	Offline target	82 chunks reassemble into seven original archives; SHA-256 verification passes.
6. Restore sandbox	Offline target	`iot-offline-build-sandbox` contains portable source/build workspaces and caches.
7. Load services	Offline target	Docker images load; Kafka storage and PostgreSQL are prepared safely; Compose starts.
8. Verify and customize	Offline target	One controlled source edit compiles with network disabled; tests pass and rollback is possible.

First-day objective: Get the two releases verified and restored. **Second-day objective:** Run a targeted offline compile. **Production objective:** Validate persistence, failover, backups, security, device hardware and an independent clean-machine restore.

Understand the architecture before installing anything

The platform joins six responsibilities: **device ingestion, MQTT routing, event streaming, data ingestion, data storage/UI, and external alerts**. It is not a single monolithic program.

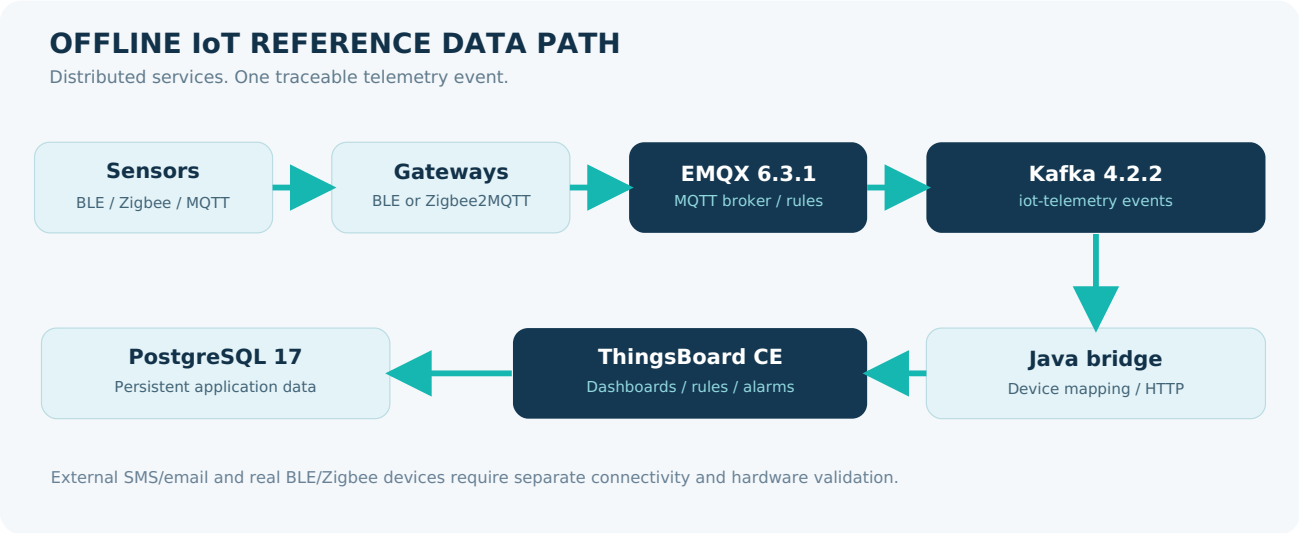


Figure 1 — Reference telemetry path. Exact MQTT topic names, credentials and mapping rules must be checked against the restored private configuration.

An example: A temperature sensor publishes `{"device":"warehouse-1","temperature":38.2}`. A Zigbee device may enter through Zigbee2MQTT; a BLE device may enter through the Python gateway; an MQTT device can publish directly. EMQX receives the MQTT message and a configured rule/sink can forward it to Kafka topic `iot-telemetry`. The Java bridge consumes the Kafka record, associates it with the correct ThingsBoard device and sends telemetry using ThingsBoard's API. ThingsBoard uses PostgreSQL for its application database and can display dashboards and alarms. SMS/email services sit outside this guaranteed message path.

Component	Pin	Primary language / job
EMQX Enterprise	6.3.1	Erlang/Elixir — MQTT broker, rule engine and MQTT-to-Kafka integration.
Apache Kafka	4.2.2	Java/Scala — event log for consumers.
ThingsBoard CE	4.3.1.6	Java + Angular/TypeScript — API, telemetry, rules and dashboards.
Zigbee2MQTT	2.14.2	Node.js/TypeScript — Zigbee coordinator gateway.
PostgreSQL	17	Database — persistent ThingsBoard application records.

Component	Pin	Primary language / job
Kafka-to-ThingsBoard bridge	Local source	Java 17 — map/forward Kafka records to ThingsBoard telemetry endpoint.
BLE-to-MQTT gateway	Local source	Python — translate BLE device events to MQTT. Physical adapter test pending.

Important distinction: Kafka is not a device UI. EMQX does not store your ThingsBoard dashboard data. ThingsBoard is not a substitute for a Kafka consumer. For language changes and alarms, prefer configuration and UI capabilities before modifying brokers.

Hardware, storage, and operating-system choices

Recommended laboratory sizing (planning guidance, not a tested minimum): x86_64 CPU, 4–8 vCPUs available to the VM, 16 GB RAM or more on the host with an appropriate VM allocation, and a **120–160 GB or larger virtual disk** for a disposable build machine. The original VM had a 78 GB Btrfs `/home` partition and used approximately 45 GB during release preparation, but additional room is needed for source changes, duplicate archives, Docker image layers and recovery.

Item	Why it matters
CPU virtualization (Intel VT-x / AMD-V)	Lets VMware run a 64-bit Linux guest reliably.
SSD / fast storage	Gradle, Maven, pnpm and tar create and read many small files. HDDs can be substantially slower.
External SSD	Better than a small FAT32 USB stick for multi-gigabyte GitHub releases.
Internet-connected preparation machine	Downloads installer packages, releases and verified Docker images before the offline test.
Reliable backup medium	The source build VM previously reported Btrfs data checksum corruption; maintain independent backup copies.
USB/Bluetooth/Zigbee access	Optional for development, but mandatory for real gateway hardware acceptance.

Disk format warning: FAT32 has a 4 GiB single-file limit. Use a suitable filesystem such as exFAT, NTFS or ext4 for large transfers, consistent with your host operating system and your security requirements. Store the 64 MiB chunk assets on the transfer medium in one directory; this avoids large per-file transfer failures.

Choose one of two deployment routes

Route A — Native openSUSE under VMware (main tutorial): Windows or Linux host → VMware VM → openSUSE Leap 16.0 → Docker Engine inside Linux. This is close to the verified development machine. Do not install Docker Desktop inside the guest.

Route B — Windows + WSL2 + Docker Desktop (legacy Source Kit): Windows host → openSUSE WSL distribution → Docker Desktop integration. This route appears in the original Source Kit manual and uses different installer steps, mounts and service management. Do not mix commands from Route A and Route B in the same running environment.

From zero: install openSUSE Leap 16.0 in VMware

This section describes a **practical setup plan**, not a claim that a brand-new VM has already passed offline restoration. Obtain the openSUSE ISO and the appropriate VMware installer from official sources while connected. Verify the ISO using the vendor-published checksum/signature. The GitHub Source Kit includes an `openSUSE-Leap-16.0.wsl` distribution asset for **WSL**, which is not a VMware bootable ISO.

1. Install and open VMware Workstation. Choose **Create a New Virtual Machine**. Select the downloaded openSUSE Leap 16.0 x86_64 installation ISO. If automatic OS detection is unavailable, choose a compatible 64-bit Linux guest.
2. Allocate RAM and processors appropriate to the host. Create a sufficiently large virtual disk, preferably on an SSD, and enable virtualization in firmware if the VM cannot start.
3. Choose virtual networking that can access your download/staging share during setup. **For the final offline test, disconnect the VM network adapter entirely**, not just one application.
4. Boot the installer, select installation destination, keyboard and timezone, and create a normal user account. Record the Linux username; the commands below use `$HOME` to avoid assuming that it is `admin`.
5. After installation, boot the VM, open a terminal and verify basic system properties before installing the development tools.

```
cat /etc/os-release
uname -m
id
pwd
lsblk -f
df -h /home
free -h
```

Expected platform for this kit: `openSUSE Leap 16.0` and `x86_64`. `lsblk -f` helps identify the filesystem, mount points and available drives. Verify the actual available disk space before copying 15+ GB of original distribution assets and 5+ GB of companion assets.

A black VMware screen is not proof that Linux is corrupted. First check VM power state, guest display settings, firmware compatibility and virtual hardware. Take a snapshot only when storage is healthy and you have an external backup; a VM snapshot is not a substitute for an independent backup.

Prepare Linux tools and the offline boundary

Docker Engine, Docker Compose v2, Git, Python 3, tar, gzip, unzip, SHA-256 tools and common build utilities are **host prerequisites**. The GitHub Docker images supply tools inside containers but do not install the Docker Engine on a new operating system.

If you are preparing the VM online, use the distribution's **configured and trusted repositories**, then verify the exact installed commands. Package names differ between openSUSE installations; inspect available packages rather than guessing blindly.

```
# Read-only discovery first:
sudo zypper repos --details
sudo zypper search docker
sudo zypper search git python3 unzip

# Example online preparation, only if these packages exist:
# sudo zypper install docker git python3 unzip curl
# Install the compatible Compose v2 package/plugin as available.

command -v docker
command -v git
command -v python3
command -v tar
command -v sha256sum
docker compose version
```

On a system without network access, stage an **offline RPM repository or verified package bundle** covering Docker Engine, Compose and their dependencies before the machine is disconnected. The two GitHub releases do not certify a complete fresh-openSUSE RPM mirror. Do not tell an offline machine to run `zypper install` against unavailable online repositories.

Once installed, start Docker with the distribution's supported service manager and verify it. If Docker access requires root, use `sudo` consistently or enroll in the Docker group after reviewing the group’s root-equivalent privileges. Do not blindly change socket permissions.

```
sudo systemctl enable --now docker
sudo docker version
sudo docker info
sudo docker compose version
```

Strict offline proof: disable the guest network adapter in VMware, then verify that Docker will not pull an image and build commands use `--network none`, `--pull never` or tool-specific offline modes. Disabling only Wi-Fi on the Windows host may not isolate a bridged or NAT virtual machine.

Know the two GitHub repositories and their releases

There are **two repositories, two release tags, and different roles**. Do not rename assets or mix the two checksums.

Repository	Release tag	Current role
`Developer20003/iot-offline-source-kit`	`v0.1.0-offline-preview`	Baseline source and runtime restoration: seven primary ZIPs; related source bundles, caches, Docker images, compiled runtime assets and 13 installer scripts.
`Developer20003/iot-offline-linux-devkit`	`v0.1.0-offline-linux-dev-preview`	Linux companion scripts/documentation, the seven prepared source/cache archives distributed as 82 upload chunks, integrity manifests and reassembly/restore scripts.

Verified release snapshot, 13 November 2026: Source Kit has **37 release assets** (approximately 15.23 GB total), including seven primary ZIP files and ancillary assets. Linux DevKit has **90 assets** (approximately 5.23 GB): 2 starter assets plus **88 added portable-environment assets**, of which 82 are data chunks. Asset counts can change if the maintainer republishes the release.

Release links (open on the Internet-connected preparation computer):

- Source Kit: `https://github.com/Developer20003/iot-offline-source-kit/releases/tag/v0.1.0-offline-preview`
- Linux DevKit: `https://github.com/Developer20003/iot-offline-linux-devkit/releases/tag/v0.1.0-offline-linux-dev-preview`

Baseline seven ZIPs (exact filenames):

Stage	Downloaded ZIP
Core/source	`01-core-and-sources.zip`
Maven/Gradle	`02-maven-gradle.zip`
Other dependencies	`03-other-dependencies.zip`
Build resources	`04-build-resources.zip`
Runtime binaries	`05-runtime.zip`
Docker images part A	`06-docker-images-a.zip`

Stage	Downloaded ZIP
Docker images part B	`07-docker-images-b.zip`

A download containing **only** these seven ZIPs may not include every stand-alone utility, WSL installer, original repository checkout or private database backup. Treat the release documentation and package checksums as the authority for the required files. The beginner procedure below assumes you also copy the original repository's installation scripts and the DevKit's release support files.

Download everything while Internet is available

Choose a **connected staging machine**. Use a trusted browser or a current GitHub CLI. The following GitHub CLI commands are an optional acquisition method, not commands for the disconnected target. They rely on `gh` being installed and authenticated if the repository requires it.

```
mkdir -p "$HOME/offline-transfer/source-release"
mkdir -p "$HOME/offline-transfer/devkit-release"

# Staging machine with Internet and GitHub CLI:
gh release download v0.1.0-offline-preview \
  --repo Developer20003/iot-offline-source-kit \
  --dir "$HOME/offline-transfer/source-release"

gh release download v0.1.0-offline-linux-dev-preview \
  --repo Developer20003/iot-offline-linux-devkit \
  --dir "$HOME/offline-transfer/devkit-release"
```

The full Source Kit release contains **37 assets**, including Windows/WSL extras. If you only need Linux, you may select a smaller documented subset, but do so only after checking which nested archives the baseline restore scripts require. For beginners, downloading the full release avoids accidentally omitting a dependency.

Acquire the **repository code** separately. The small Linux DevKit source `.tar.gz` release asset and the Source Kit repository checkout contain scripts; the release's data chunks alone do not contain the entire user-facing repository structure. A connected machine can clone the repositories and copy the checkouts (without secrets) to external media, or download their source archives at the pinned tag.

```
# Connected staging machine, example:
git clone https://github.com/Developer20003/iot-offline-source-kit.git
git clone https://github.com/Developer20003/iot-offline-linux-devkit.git

# Optional: inspect their branch/tag state and copy the result
# to your external drive after verifying no private files were added.
```

Keep the downloads organized: one directory for original release ZIPs, one directory for Linux DevKit `.chunk-*` files and the `SHA256SUMS-CHUNKS` manifest, plus one copy of each source repository. Avoid putting the 82 DevKit chunks into nested folders unless you also update the reassembly script's expected location.

Verify downloads, the USB transfer, and archive integrity

No download or transfer should be trusted only because the filename looks correct. Before moving to the offline computer, check that downloads completed, that the transfer filesystem supports the file sizes, and that available SHA-256 manifests cover the actual files.

```
# Linux staging computer - read-only checks:
cd "$HOME/offline-transfer/source-release"
ls -lh 0*.zip
unzip -t 01-core-and-sources.zip

cd "$HOME/offline-transfer/devkit-release"
ls -1 *.chunk-* | wc -l  # Expected 82 for published snapshot.
sha256sum -c SHA256SUMS-CHUNKS
```

The DevKit manifest is relative: run `sha256sum -c SHA256SUMS-CHUNKS` from the same folder containing the 82 flat chunk files. Do not run it from the repository root while the chunks are in another folder.

Original Source Kit checksums are supplied as `RELEASE_SHA256SUMS` and `SHA256SUMS` assets and within nested archives. Inspect the manifest format before running it; paths may assume a particular extraction layout. If a checksum fails, **replace the bad download from the connected computer** instead of continuing.

```
# Show top-level ZIP structure before selecting extraction root:
unzip -Z1 01-core-and-sources.zip | head -30

# After copying to the target, repeat verification there.
# Prefer a second independent checksum comparison after USB transfer.
```

Do not disable checksum validation to make an install work. A valid checksum proves the downloaded bytes match the manifest; it does not prove no secrets were included, that the software is suitable for production, or that a broken Btrfs filesystem has healed.

Restore the original Source Kit baseline

The baseline contains the source and runtime package required by the companion DevKit's restore script. **Restore it first** into `$HOME/projects/iot-offline-development`, preserving the downloaded distribution separately.

A. Put the original repository and ZIP contents together

Copy the `iot-offline-source-kit` checkout or extracted repository source to the target. Inspect each of the seven ZIPs before extraction because the appropriate extraction directory depends on the archive's internal paths. The expected finished source-kit root contains `offline-bundle/`, `scripts/`, and `README.md`; it is not seven unrelated ZIP directories.

```
mkdir -p "$HOME/projects"
cd "$HOME/projects/iot-offline-source-kit"

# Inspect first; never extract blindly over live data:
ls -la
ls -la offline-bundle scripts
ls -lh /path/to/source-release/0*.zip
unzip -Z1 /path/to/source-release/01-core-and-sources.zip | head -25
```

Extract the **seven ZIPs according to their internal paths** into a clean staging location, then confirm that the nested `offline-bundle/git`, `offline-bundle/dependencies`, `offline-bundle/build-resources`, `offline-bundle/runtime`, `offline-bundle/config`, and `offline-bundle/docker-images` directories match the restore script layout. Do not extract if a ZIP unexpectedly contains absolute paths or parent-directory traversal. The earlier Source Kit manual documents the original seven-ZIP distribution; its exact folder layout should be checked against your downloaded archive revision.

B. Verify installer scripts and all nested assets

If the repository already contains the `scripts/` folder, **do not run** `bootstrap-offline.sh --extract`, because it refuses to overwrite existing scripts. Use the read-only verification mode. Use `--extract` only when the installer archive is present and the scripts directory genuinely does not yet exist.

```
cd "$HOME/projects/iot-offline-source-kit"

bash offline-bundle/bootstrap-offline.sh --verify
bash scripts/restore-complete-offline-kit.sh --verify
```

The expected final verification line from the source-kit script is `COMPLETE OFFLINE PACKAGE VERIFICATION: PASS`. If it reports a missing artifact, stop and fix the distribution layout or missing download; do not copy random files into runtime folders.

C. Restore an independent baseline development kit

```
cd "$HOME/projects/iot-offline-source-kit"

# The destination must NOT already exist:
test ! -e "$HOME/projects/iot-offline-development"

bash scripts/restore-complete-offline-kit.sh \
  --restore "$HOME/projects/iot-offline-development"
```

The restore script creates independent source worktrees, caches, build resources, runtime assets and fresh private configuration files. It checks pinned source commits and restores 13 installation scripts. **It does not automatically load Docker images, initialize Kafka, restore PostgreSQL, or start containers.** These are later, separate steps.

D. Check that the baseline directory exists

```
BASE="$HOME/projects/iot-offline-development"

test -d "$BASE/sources/emqx"
test -d "$BASE/sources/kafka"
test -d "$BASE/sources/thingsboard"
test -d "$BASE/sources/zigbee2mqtt"
ls -ld "$BASE" "$BASE/caches" "$BASE/runtime"
```

If you already restored the baseline on a previous day: do not rerun `--restore` over it. The script deliberately refuses to overwrite an existing target. Verify the existing kit and move on to the DevKit chunks.

Understand the companion Linux DevKit contents

The Linux companion has two layers:

1. The **small Git repository / starter asset**: developer scripts, bridge and gateway source, build/test documentation, configuration templates and patch storage.
2. The **portable build environment**, uploaded after the original guide: seven prepared archives split into 82 smaller GitHub assets, plus reassembly and restore scripts and checksum manifests.

The large archives correspond to these logical workspaces:

Portable archive	What it restores
`01-emqx-workspace.tar.gz.part-001`	EMQX Enterprise source, prepared dependencies and prior build artifacts.
`02-kafka-workspace.tar.gz.part-001`	Kafka isolated source and build workspace.
`03-thingsboard-workspace.tar.gz.part-001`	ThingsBoard isolated source and compiled/backend/frontend workspace.
`04-zigbee-workspace.tar.gz.part-001`	Zigbee2MQTT isolated source and packages.
`05-gradle-cache.tar.gz.part-001`	Gradle 9.2.1 / Kafka offline build cache.
`06-maven-cache.tar.gz.part-000`	ThingsBoard offline Maven dependency cache.
`07-pnpm-cache.tar.gz.part-000`	Offline pnpm package store for Zigbee2MQTT.

Why 82 chunks instead of seven large uploads? The first 889 MB asset upload failed when the network connection reset at about 14.8%. Uploading as 64 MiB chunks succeeded, and GitHub confirmed all 90 assets were in the `uploaded` state. The chunk files are transport segments; they are **not** meant to be extracted independently.

The new GitHub release description may still contain early preview wording about caches being absent from the starter source archive. That remains true of the tiny starter `.tar.gz`; the caches now exist as separate published portable chunk assets.

Reassemble the 82 chunks and verify all seven archives

Perform this step **after** baseline restoration, using the companion release files from the external drive. Place the **82 data chunks** and the following support files in the **same directory**:

`SHA256SUMS-CHUNKS`, `SHA256SUMS-PORTABLE`, `SOURCE-REVISIONS.txt`, `INSTALL.txt`, `assemble-portable-chunks.sh`, and `restore-portable-environment.sh`.

```
mkdir -p "$HOME/projects/devkit-portable-download"
# Copy the DevKit release chunks and 6 support files here,
# using your verified external-drive mount.

cd "$HOME/projects/devkit-portable-download"

# Count and verify the downloaded 64 MiB assets:
find . -maxdepth 1 -type f -name '*.chunk-*' | wc -l
sha256sum -c SHA256SUMS-CHUNKS

# Assemble the original seven archives in this directory:
bash assemble-portable-chunks.sh
```

Expected final line: `PORTABLE_ARCHIVES_REASSEMBLED=PASS`. The reassembly script concatenates the chunks in numeric filename order, generates each original `.tar.gz.part-*` file and verifies it against `SHA256SUMS-PORTABLE`. **Do not manually unpack the `.chunk-` files with tar.**

Space planning: Reassembly adds another full copy of the seven compressed archives alongside the chunks. Keep enough free space for both sets before starting, then additional space for the extracted workspaces. Do not delete the original chunks until the restored machine has been tested and you have independent recovery media.

Restore the portable Linux build workspace

Once the baseline exists and the seven archives have been reassembled, the companion restore script can create the independent development sandbox. It expects the default base directory under `$HOME/projects` unless you set `OFFLINE_PROJECTS_DIR`.

```
cd "$HOME/projects/devkit-portable-download"

# Baseline prerequisite:
test -d "$HOME/projects/iot-offline-development/sources"

# The sandbox must NOT already exist:
test ! -e "$HOME/projects/iot-offline-build-sandbox"

# Restores all seven verified archives under ~/projects:
bash restore-portable-environment.sh
```

The script checks `SHA256SUMS-PORTABLE`, restores four prepared source workspaces and three dependency caches, then verifies Kafka, ThingsBoard and Zigbee2MQTT Git commit IDs and the EMQX `mix.exs` source file. Expected final message: `PORTABLE_OFFLINE_ENVIRONMENT_RESTORED=PASS`.

```
DEV="$HOME/projects/iot-offline-build-sandbox"

du -sh "$DEV"/*
ls -ld "$DEV/emqx-6.3.1"
ls -ld "$DEV/isolated-sources/"*
ls -ld "$DEV/isolated-caches/"*
```

Two critical limitations: (1) Restoration was verified while creating the release archives, but the newly published package has **not yet** completed a clean-machine restore on a second laptop. (2) Restoring precompiled artifacts does not mean a clean full source build is reproducible; a changed-file compile acceptance test is required.

If a script says the sandbox already exists, **do not delete it automatically**. Preserve or back up the existing work first. For a new test, use a fresh VM or a new verified target location, remembering that the script's checksum and directory assumptions must still match.

Load Docker images and prepare the baseline runtime

The baseline Source Kit's Docker image archives provide the tested container toolchains. They are a **different artifact set** from the companion's Gradle/Maven/pnpm caches. The image-loader script verifies the local archive checksums before running `docker image load`.

```
PKG="$HOME/projects/iot-offline-source-kit"
BASE="$HOME/projects/iot-offline-development"

# Check that Docker Engine is available first:
sudo docker info

# Verify original Docker image archives:
bash "$PKG/scripts/restore-offline-docker-images.sh" --verify

# Load them once; this requires no registry access:
bash "$PKG/scripts/restore-offline-docker-images.sh" --load

# Inspect local images:
sudo docker images
```

Expected image references include `postgres:17`, `eclipse-temurin:17-jdk`, `maven:3.9.9-eclipse-temurin-17`, the pinned EMQX builder image, `node:22.22.2-bookworm`, and `python:3.12`. **Use** `docker image inspect` **before invoking a builder with** `--pull never`. If an image is missing, return to the offline image archive, not a live registry.

Read-only runtime preparation check

```
bash "$PKG/scripts/prepare-restored-offline-kit.sh" --verify
sudo docker compose -f "$BASE/compose.yaml" config --quiet
```

The `--prepare` mode performs important stateful operations: loads images, creates a managed Docker network and PostgreSQL volume, and initializes **new** Kafka KRaft storage. Run it only on a **fresh, restored kit with no conflicting containers or existing data**.

```
# Fresh target only, after reviewing the verification output:
bash "$PKG/scripts/prepare-restored-offline-kit.sh" \
  --prepare "$BASE"
```

The preparation script checks for conflicts named `iot-postgres`, `iot-emqx`, `iot-kafka` and `iot-thingsboard`. It refuses to overwrite unmanaged Docker network/volume resources. Kafka's `meta.properties` is stateful; once written, **never regenerate KRaft storage** just to solve a startup error. Record its current cluster ID and preserve the data directory.

PostgreSQL: distinguish new installation from backup restoration

This is a major prerequisite that beginners frequently miss. The public release does **not** contain a private ThingsBoard PostgreSQL database dump. The repository's tested baseline restoration path expects a separately supplied, approved PostgreSQL 17 `pg_dump` custom-format backup. Its supplied restore script refuses to write to a nonempty database volume, preventing accidental overwrites.

Route A — Existing approved database backup (documented script path)

1. On the authorized existing machine, create and securely retain an application database backup according to your operational backup policy. Validate it and transfer it on encrypted or otherwise protected storage. **Do not upload it to GitHub**.
2. Ensure the new target was prepared with the source kit's `--prepare` script and that the PostgreSQL managed volume is unused and empty.
3. Verify the dump with the repository script, then restore it into the fresh PostgreSQL volume.

```
PKG="$HOME/projects/iot-offline-source-kit"
BASE="$HOME/projects/iot-offline-development"
PRIVATE_DUMP="/path/to/authorized-backup.dump"

bash "$PKG/scripts/restore-offline-postgres.sh" \
  --verify-backup "$PRIVATE_DUMP"

# Only on a new, empty, managed PostgreSQL volume:
bash "$PKG/scripts/restore-offline-postgres.sh" \
  --restore "$BASE" "$PRIVATE_DUMP"
```

Expected message after a successful, populated restore: `OFFLINE POSTGRESQL DATABASE RESTORATION: PASS`. The script tests the archive with `pg_restore --list`, starts a temporary isolated PostgreSQL container, restores tables, verifies table count and removes the temporary container.

Credentials caution: The restored kit generates a new `POSTGRES_PASSWORD`; the existing ThingsBoard environment file must use compatible database credentials. Inspect the `SPRING_DATASOURCE_*` configuration and the restored dump's database/role assumptions before starting services. Do not print secrets in screenshots or command logs.

Route B — Brand-new empty database (requires separate validation)

If you truly start from zero and have **no private backup**, a fresh ThingsBoard schema/database initialization process is needed. The public baseline `restore-offline-postgres.sh` does **not** implement the fresh schema installer. A candidate procedure must use the correct ThingsBoard

4.3.1.6 application installer and its bundled data/SQL, verify Java and database settings, and run **only on a new, empty volume**. It must be tested in a disposable VM first. This manual intentionally does not invent an unverified command that might damage an existing database.

Decision: If Route A applies, proceed. If Route B applies, treat fresh schema initialization as an **open installation gate**, not an invisible prerequisite. You can still restore sources, caches and build offline without starting the ThingsBoard runtime yet.

Start services safely and perform first smoke tests

Once PostgreSQL has been initialized/restored, check the baseline startup preflight. Starting services may create runtime state; it is different from a read-only audit.

```
PKG="$HOME/projects/iot-offline-source-kit"
BASE="$HOME/projects/iot-offline-development"

# Validate the platform before starting:
bash "$PKG/scripts/start-offline-services.sh" \
  --verify "$BASE"

# Fresh target only, after database preparation:
bash "$PKG/scripts/start-offline-services.sh" \
  --start "$BASE"

# Check services; Docker access may require sudo:
sudo docker ps -a
sudo docker compose -f "$BASE/compose.yaml" ps
```

Expected core containers: `iot-postgres`, `iot-emqx`, `iot-kafka`, and `iot-thingsboard`. **Do not infer a service is healthy only from a running container.** Inspect its logs, readiness endpoint or functional test.

```
# Diagnostic examples; these only read status and logs:
sudo docker logs --tail=80 iot-postgres
sudo docker logs --tail=80 iot-emqx
sudo docker logs --tail=80 iot-kafka
sudo docker logs --tail=80 iot-thingsboard

# Inspect exposed ports without assuming port numbers:
sudo docker ps --format 'table {{.Names}}\t{{.Ports}}'
ss -lnt
```

If the restored Compose file publishes ThingsBoard on local port 8080, a simple HTTP check can be:

```
curl -I --max-time 10 http://127.0.0.1:8080/
```

If this fails, check the actual Compose port mapping before concluding the application is down. EMQX's typical MQTT and dashboard ports and Kafka listeners are configurable; consult the restored configuration rather than hard-coding assumptions.

Confirm the application path in layers

1. **PostgreSQL:** container ready, expected application tables present; no unwanted installer reruns.
2. **EMQX:** MQTT listener reachable, test publisher and subscriber exchange a known, non-sensitive message.

3. **Kafka:** topic exists, producer/consumer can exchange one known event; inspect listeners and advertised address.
4. **EMQX → Kafka:** check rule `offline_mqtt_to_kafka`, connector `offline_kafka_connector`, sink `offline_telemetry_sink` and topic `iot-telemetry` against the restored environment. These names were verified on the original VM, but fresh-VM provisioning may differ.
5. **Bridge → ThingsBoard:** Java bridge has a valid device mapping and an authorized ThingsBoard token; check HTTP response and Latest Telemetry for the device. A prior HTTP acknowledgement does not by itself prove persistence on a new installation.
6. **Dashboards/alerts:** open the ThingsBoard UI from a browser that can access the isolated lab LAN, create a disposable test dashboard and verify a test alarm.

Expected test message (illustrative, not a secret-bearing production schema):

```
{"device":"lab-sensor-01","temperature":23.7,"ts":1763000000000}
```

Do not send arbitrary telemetry to a production topic. Use a dedicated test topic/device and confirm the bridge's expected JSON field names before publishing.

Directory map: know exactly what you can edit

These are the **three important directories** on the new Linux laptop. Keep them separate.

```
~/projects/
  iot-offline-source-kit/        # original source repository + release assets
  iot-offline-development/      # restored baseline source and runtime kit
    sources/                     # EMQX, Kafka, ThingsBoard, Zigbee2MQTT
    caches/                      # original restore caches
    runtime/                     # persistent config/data: protect
    compose.yaml                 # baseline service layout
    scripts/                     # baseline install/start scripts
  iot-offline-linux-devkit/     # companion source/scripts/docs repository
  iot-offline-build-sandbox/    # new prepared isolated build workspaces
    emqx-6.3.1/
    isolated-sources/kafka/
    isolated-sources/thingsboard/
    isolated-sources/zigbee2mqtt/
    isolated-caches/gradle/
    isolated-caches/maven/
    isolated-caches/pnpm/
  devkit-portable-download/     # 82 chunks + assembled originals + manifests
```

Edit source files under `iot-offline-build-sandbox` for compilation experiments, not inside a running container volume. Keep sensitive runtime configuration under the baseline runtime tree with appropriate filesystem permissions. The `iot-offline-linux-devkit` Git repository should hold authored scripts, documentation, small bridge/gateway source and patches, not giant caches or live database contents.

```
# Read-only orientation commands:
BASE="$HOME/projects/iot-offline-development"
DEV="$HOME/projects/iot-offline-build-sandbox"

find "$BASE/sources" -maxdepth 2 -type d | head -30
find "$DEV/isolated-sources" -maxdepth 2 -type d | head -30

git -C "$DEV/isolated-sources/kafka" rev-parse HEAD
git -C "$DEV/isolated-sources/thingsboard" rev-parse HEAD
```

The original source revisions must stay pinned to the packaged caches. Updating Kafka, EMQX, ThingsBoard or Zigbee2MQTT may require acquiring a new dependency set on an Internet-connected build-preparation computer.

Git and source editing: a safe beginner workflow

Use one change, one component, one branch, one proof. Do not mix a ThingsBoard translation edit, an EMQX rule change and a Kafka version upgrade in the same first experiment.

Record the baseline first

```
DEV="$HOME/projects/iot-offline-build-sandbox"
KAFKA="$DEV/isolated-sources/kafka"

# Confirm the source identity and current changes:
git -C "$KAFKA" rev-parse HEAD
git -C "$KAFKA" status --short

git -C "$KAFKA" switch -c rjh-lab-first-change
```

If a restored Git tree lacks a local configuration/remote, `git status` may work even while remote operations do not. A missing remote is not evidence that source code is incomplete; record and verify the exact local commit first. **Do not run `git reset --hard`, `git clean -fdx`, or checkout a different tag** in a prepared workspace unless you have verified backups, because these actions can destroy build caches and local work.

Make and review the change

Edit a **source-controlled, low-risk target** appropriate for that component, not a binary or compiled `.class` / `.beam` output. Use an editor such as VS Code, Vim or Nano. Before and after a change:

```
git -C "$KAFKA" diff --stat
git -C "$KAFKA" diff --check
git -C "$KAFKA" status --short
```

For a first offline rebuild acceptance test, a harmless comment edit in an actual source file may be insufficient to demonstrate compiler work if the build tool considers nothing changed. Prefer a small, reversible source change that triggers recompilation, then confirm the build task log reports relevant compilation.

Save authored changes cleanly

```
# Stage only the intended tracked file(s):
# git -C "$KAFKA" add path/to/changed-file
# git -C "$KAFKA" commit -m 'Lab: validate offline build'

# For portable changes, export verified commits as patches:
# git -C "$KAFKA" format-patch <pinned-base-sha> \
#   -o "$HOME/projects/iot-offline-linux-devkit/patches/kafka"
```

Commit source changes to the correct project's Git repository. Keep large build outputs in release assets. Before pushing any changes, scan for private device tokens, `*.env`, SSH keys, passwords, proprietary customer data and logs.

How strict offline build verification works

An “offline build” is not a vague statement. Each successful acceptance test must demonstrate:

- A source tree at the expected pinned Git revision, with the intended change documented.
- Compiler and build dependencies available from the local cache or a loaded builder image.
- Network access denied to build containers (`--network none`) and tools configured in offline mode (`-o`, `--offline`, `--frozen-lockfile`, etc. as appropriate).
- A command returning exit code zero and producing the expected artifacts.
- A test proving the change was compiled, not merely that an old artifact still existed.
- No undocumented runtime modifications, data loss, new secret exposure or unapproved Internet download.

Component	Previously verified level	Important boundary
EMQX	Offline targeted Erlang/Elixir module recompilation	**Not** a clean entire enterprise release build.
Kafka	Gradle 9.2.1 `jar`, 169 tasks	Prepared cache and source used.
ThingsBoard backend	Maven `application -am compile`	Not a complete installable distribution package.
ThingsBoard Angular UI	`npm run build:prod`	Uses prepared `node_modules`.
Zigbee2MQTT	Offline pnpm install plus TypeScript build	Install scripts skipped; lockfile trust revalidation not performed.
Java bridge	Offline Java compilation	Production retry/idempotency and multi-device durability pending.

Build logs are evidence. For each command capture date, image/version, source SHA, network-isolation method, exit code, elapsed time, exact artifact location and whether a modified source file actually rebuilt.

```
# Template to capture logs for a READ-ONLY check:
mkdir -p "$HOME/projects/offline-evidence"
date -Is | tee "$HOME/projects/offline-evidence/start.txt"
# Put your actual build command in the next step-specific chapter.
```

EMQX Enterprise 6.3.1: edit and targeted recompilation

EMQX uses Erlang/Elixir with a **project-specific builder**. Do not substitute an arbitrary system Erlang package. The verified original builder was:

```
ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04
```

Its prepared workspace is:

```
~/projects/iot-offline-build-sandbox/emqx-6.3.1
```

Step A: verify builder and workspace before editing

```
DEV="$HOME/projects/iot-offline-build-sandbox"
EMQX="$DEV/emqx-6.3.1"
IMAGE='ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04'

sudo docker image inspect "$IMAGE" >/dev/null
ls -l "$EMQX/mix.exs"
ls -ld "$EMQX/deps" "$EMQX/_build"
git -C "$EMQX" rev-parse HEAD
```

Expected pinned source commit: `b0604926195f9372b0068b61f88d409d639a2401`. The earlier VM passed **module-level recompilation** within the prepared workspace using `./mix compile --no-deps-check --no-prune-code-paths`. Recreate the exact command in the container that has the required toolchain and locally prepared source; this is **not** evidence of a clean `make` enterprise build.

Step B: targeted offline compile — only after preflight

```
# Candidate invocation matching the verified module-compile method.
# Verify ./mix exists and is executable before using it.
sudo docker run --rm --network none --pull never \
  -v "$EMQX:/emqx" -w /emqx \
  --entrypoint /bin/bash "$IMAGE" \
  -lc 'test -x ./mix && ./mix compile \
       --no-deps-check --no-prune-code-paths'
```

If permission or SELinux labeling fails, do not silently edit host ownership or relabel a production runtime mount. Confirm UID/GID and use appropriate mounts only in disposable workspaces. `:z` may be required on SELinux hosts, but it changes host file labels and is not a universal safety default.

Step C: distinguish module build from full release build

A full EMQX Enterprise distribution may require the exact forked Erlang/OTP, Hex/Mix/Rebar3 archives and native dependencies. A clean full build has **not** been certified by these assets. If the build tries to download a missing Hex dependency, stop and record it. Acquire the correct dependency from an online staging computer, version it in a future kit and retest offline.

License: EMQX 6.3.1 source is under Business Source License 1.1. The pinned license permits copying and redistribution under its terms, but hosted or embedded commercial production use has restrictions. Retain copyright/license notices and review all transitive dependency licenses before redistributing a changed release.

Apache Kafka 4.2.2: an offline Java/Scala build

Kafka uses Gradle. The verified original VM completed the offline `jar` workflow using **Gradle 9.2.1**, with **169 tasks**. The prepared source and cache are:

```
~/projects/iot-offline-build-sandbox/isolated-sources/kafka
~/projects/iot-offline-build-sandbox/isolated-caches/gradle
```

Step A: inspect the verified environment

```
DEV="$HOME/projects/iot-offline-build-sandbox"
KAFKA="$DEV/isolated-sources/kafka"
GRADLE_CACHE="$DEV/isolated-caches/gradle"

ls "$KAFKA" | head -20
ls -ld "$GRADLE_CACHE"
git -C "$KAFKA" rev-parse HEAD
find "$DEV" -path '*/gradle-9.2.1/bin/gradle' -print
```

Expected source commit: `c5e942859576203e1b00260070fe6241b64db251`. Locate the Gradle executable before copying a command from another machine: the verified VM had a local Gradle 9.2.1 installation under its prepared working tree. Do not assume the wrapper can download a distribution while offline.

Step B: offline jar build

```
# Run from the Kafka source directory. Choose whichever Gradle
# 9.2.1 executable is actually available in your restored kit.
cd "$KAFKA"
export GRADLE_USER_HOME="$GRADLE_CACHE"

# If the local Gradle executable is at ./gradle-9.2.1/bin/gradle:
./gradle-9.2.1/bin/gradle \
  --offline --no-daemon jar
```

If the local executable is elsewhere, replace **only the executable path**, not the pinned Gradle version or `--offline` flag. Run in an offline builder/container if that is where the correct Java JDK is staged, with network disabled.

Step C: validate actual output

Look for newly compiled Kafka client/server artifacts and the Gradle `BUILD SUCCESSFUL` summary. Examine `build/libs` directories relevant to the changed module. **Do not delete or overwrite the live Kafka distribution** until you have a staged integration test and rollback copy.

Common failure: `No cached version available for offline mode`. That means a cache miss, not a reason to enable the network on the offline target. Package the missing Gradle module/plugin on a connected staging machine and record its version and checksum.

ThingsBoard backend 4.3.1.6: Maven offline compile

ThingsBoard CE combines a Java backend with a separate Angular frontend. They are **two build workflows**, not one. The original offline VM successfully compiled the Maven reactor through the `application` module with `-am`. It did **not** certify the complete deployment package.

```
Source:  ~/projects/iot-offline-build-sandbox/isolated-sources/thingsboard
Cache:   ~/projects/iot-offline-build-sandbox/isolated-caches/maven
Commit:  c77335f4a524bb9fe0209806672c6ec7aea76141
```

Step A: inspect the cache layout

```
DEV="$HOME/projects/iot-offline-build-sandbox"
TB="$DEV/isolated-sources/thingsboard"
MVN="$DEV/isolated-caches/maven"

ls -l "$TB/pom.xml"
find "$MVN" -maxdepth 2 -type d | head -25
find "$MVN" -name 'protoc-3.25.5-linux-x86_64.exe' -print
```

The previous damaged artifact was `com/google/protobuf/protoc/3.25.5/protoc-3.25.5-linux-x86_64.exe` in the Maven cache. It was replaced from the official repository and passed its published SHA-1 checksum, after which Maven cache packaging succeeded. The file repair **did not certify the whole Btrfs disk**.

Step B: isolate the build and compile

The **verified Maven argument sequence** on the original VM was:

```
mvn -o -B -ntp \
  -Dmaven.repo.local=/offline-maven \
  -Dmaven.test.skip=true \
  -Dskip.yarn=true \
  -Dskip.ui.build=true \
  -Dpkg.skip=true \
  -Dlicense.skip=true \
  -pl application -am compile
```

Run it only from the ThingsBoard source directory in the prepared Maven/JDK 17 environment with a real local Maven repo mounted at `/offline-maven`. The restored cache may have a nested `repository/` subdirectory: **confirm the directory structure before selecting the host mount**, so Maven actually sees `com/`, `org/` and other package paths under `/offline-maven`.

```
# Illustration of the required mount mapping; validate on your machine.
# Do not execute until the expected cache directory is confirmed.
# sudo docker run --rm --network none --pull never \
#   -v "$TB:/thingsboard" -w /thingsboard \
#   -v "$MVN/repository:/offline-maven" \
#   --entrypoint /bin/bash maven:3.9.9-eclipse-temurin-17 \
#   -lc 'mvn -o -B -ntp ... -pl application -am compile'
```

The precise previous pass was an offline **compile** of the application and its Maven reactor dependencies. Packaging a ThingsBoard boot JAR for deployment is a separate build/acceptance exercise. Do not put a newly compiled class directory into the live service without staged integration testing.

ThingsBoard Angular UI: language changes and production build

The frontend is under `ui-ngx/` in the pinned ThingsBoard source. The original VM passed an **offline production frontend bundle** with Node `22.22.2-bookworm` and prepared `node_modules`, using:

```
cd "$HOME/projects/iot-offline-build-sandbox/isolated-sources/thingsboard/ui-ngx"

# Run inside the prepared compatible Node environment:
npm run build:prod
```

Change UI text safely

1. In ThingsBoard itself, first check whether changing the logged-in user's language already meets the requirement. No source rebuild is necessary for a built-in locale choice.
2. For a wording correction or custom translation, locate the actual locale assets in the pinned source, such as `ui-ngx/src/assets/locale/`; inspect the source before editing.
3. Create a branch, copy the target file for comparison, edit the text without changing message keys or placeholders, and validate its JSON syntax.
4. Build the UI using the prepared offline Node environment, then review the resulting bundle in a **staged** ThingsBoard application.

```
TB="$HOME/projects/iot-offline-build-sandbox/isolated-sources/thingsboard"
find "$TB/ui-ngx/src/assets/locale" -maxdepth 1 -type f | head -20

# Check JSON syntax for one selected file:
# python3 -m json.tool "$TB/ui-ngx/src/assets/locale/<file>.json" >/dev/null
```

Preserve dynamic placeholders and localization keys. A translation key such as `{0}` or `{{value}}` may be consumed by application code. Rewriting it as prose can silently break the UI. Test menus, validation errors, dashboard text and formatting of dates/units, not just the login page.

Zigbee2MQTT 2.14.2: TypeScript and pnpm offline build

The Zigbee2MQTT workspace uses a prepared pnpm package store and Node environment. The original VM reported `pnpm 11.26.0` in its local offline tooling and passed TypeScript compilation after an offline package install. The pinned source is `344b6e03b43644c97154a72ca9eae17193c553a2`.

```
Source:  ~/projects/iot-offline-build-sandbox/isolated-sources/zigbee2mqtt
Cache:   ~/projects/iot-offline-build-sandbox/isolated-caches/pnpm
```

Step A: inspect the prepared offline toolchain

```
DEV="$HOME/projects/iot-offline-build-sandbox"
Z2M="$DEV/isolated-sources/zigbee2mqtt"
PNPM="$DEV/isolated-caches/pnpm"

ls -l "$Z2M/package.json" "$Z2M/pnpm-lock.yaml"
find "$PNPM" -name 'pnpm.cjs' -print | head -10
ls -ld "$PNPM/store"
```

Step B: reproduce the tested isolated workflow

The verified VM used a local pnpm executable and these arguments:

```
# Execute from the Zigbee2MQTT source inside a prepared Node image.
# Confirm the pnpm path and store mount before execution.
pnpm install --offline --frozen-lockfile \
  --ignore-scripts --config.trust-lockfile=true \
  --store-dir=/pnpm/store

pnpm run build
```

`--ignore-scripts` avoids package install scripts; `--config.trust-lockfile=true` bypasses some online trust/revalidation behavior. These flags explain the **specific verified test**, not a recommendation to disable future security verification. Recheck package integrity while connected during release preparation.

Hardware integration is a different test

A simulated build does not confirm USB coordinator ownership, serial permissions, paired Zigbee devices or over-the-air behavior. For real testing, determine the actual coordinator path and type, map the host USB device into the Zigbee2MQTT container, back up `configuration.yaml` and the device database, and test on an isolated network. Do not reset pairing or erase device state as part of a source-build test.

The Java Kafka-to-ThingsBoard bridge

The bridge is authored application code, separate from Kafka itself. It reads `iot-telemetry`, interprets the event, selects a ThingsBoard device credential and calls ThingsBoard's HTTP telemetry API. This is a good first customization target because a change does **not** require rebuilding Apache Kafka.

The Linux companion repository contains:

```
gateway/offline-kafka-thingsboard/src/OfflineKafkaThingsBoardBridge.java
scripts/import-local-bridge.sh
scripts/build-offline-bridge.sh
compose/bridge.compose.yaml
```

The original VM passed an offline Java source compilation and a selected MQTT → EMQX → Kafka → ThingsBoard HTTP forwarding test. But the bridge was previously stopped during part of verification, and durable multi-device production semantics remain unproven.

Safe build procedure

```
DEVKIT="$HOME/projects/iot-offline-linux-devkit"

cd "$DEVKIT"

# Inspect before overwriting any local authored source:
ls -l gateway/offline-kafka-thingsboard/src/

# If needed, use the documented import script carefully:
# bash scripts/import-local-bridge.sh

# Build in the offline Java environment:
bash scripts/build-offline-bridge.sh
```

Do not launch a second bridge automatically. The companion Compose template is a proposed migration path; running two consumers with accidental group/client conflicts can cause duplicate delivery or other unintended Kafka offset behavior. A safe migration requires state backup, stopping the previous bridge, validating the new Compose configuration and confirming exactly one intended consumer instance.

Production hardening checklist

- Keep a protected `device_id -> ThingsBoard device token` mapping outside Git and logs.
- Validate event schema, timestamps, value types and allowed device identities.
- Bound HTTP retries with backoff; avoid infinite tight loops.
- Decide when to commit Kafka offsets; document the acknowledged-delivery boundary.
- Handle duplicate Kafka records and ThingsBoard writes by a defensible idempotency/deduplication policy.

- Add observability, dead-letter handling, restart recovery and tests for invalid tokens/unavailable ThingsBoard.

Example transport record, without a private token:

```
{
  "device_id": "lab-sensor-01",
  "timestamp_ms": 1763000000000,
  "telemetry": {"temperature": 23.7, "humidity": 44.1}
}
```

Schema note: This is a design example. The actual bridge's accepted field names must be confirmed by reading its Java source; do not assume that it already accepts this exact structure.

The Python BLE-to-MQTT gateway

BLE means **Bluetooth Low Energy**, not Zigbee. A Python gateway scans or connects to BLE peripherals, translates their readings into JSON and publishes to EMQX over MQTT. It needs a real Bluetooth adapter and Linux permissions on the target machine to perform hardware tests.

The original VM verified two Python unit/simulation tests and published a simulated BLE telemetry message to EMQX with MQTT QoS 1/PUBACK. **There was no real host Bluetooth hardware in that VM** (`/sys/class/bluetooth` was empty). A successful mock test is not a device interoperability test.

```
# Locate the authored gateway source in the restored repositories:
find "$HOME/projects/iot-offline-linux-devkit" \
  -maxdepth 4 -type f \( -name '*ble*py' -o -name 'gateway.py' \)

# Check host Bluetooth hardware (read-only):
lsusb
ls /sys/class/bluetooth 2>/dev/null || true
systemctl status bluetooth --no-pager || true
```

A real-device test requires an adapter visible inside Linux, a known compatible BLE device, approved pairing credentials if needed, matching Python packages (e.g., BLE and MQTT client packages from an offline wheelhouse), correct USB passthrough/udev permissions, and an MQTT broker reachable on the private LAN.

Offline Python dependencies: if a required wheel is missing, pre-download the correct wheel set **for the target Linux architecture and Python version** on the connected staging machine, record hashes, then install without index access. Do not assume a Windows wheel works on openSUSE Linux.

```
# Example pattern, only after preparing an approved wheelhouse:
# python3 -m venv "$HOME/projects/ble-venv"
# "$HOME/projects/ble-venv/bin/pip" install --no-index \
#   --find-links /path/to/verified-wheelhouse -r requirements.txt
```

For gateway acceptance, test scan/connection, expected payload decoding, reconnect after adapter removal, MQTT QoS 1 acknowledgement, and event arrival in ThingsBoard. Separately verify **data persistence and duplicate handling**; neither is guaranteed by a successful PUBACK alone.

Customize MQTT routing, Kafka topics, dashboards, and alerts

Most application changes can be done **without changing broker source code**. Start with configuration and use a staged test device.

User goal	First place to change	Usually requires source rebuild?
Route device telemetry to another Kafka topic	EMQX rule/connector/sink and bridge consumer settings	No
Rename a device or dashboard label	ThingsBoard UI/entity settings	No
Add an alarm threshold	ThingsBoard rule chain / alarm configuration	No
Add a device-token mapping	Java bridge protected mapping/config	May require bridge rebuild or restart
Fix a translation	ThingsBoard locale resource	Usually UI build
Implement a new unsupported MQTT broker behavior	EMQX Erlang/Elixir source	Yes
Change Kafka protocol/internals	Kafka Java/Scala source	Yes
Add Zigbee converter behavior	Zigbee2MQTT converter/config/ TypeScript	Maybe

Example: test a temperature alarm

1. Create a disposable `lab-sensor-01` ThingsBoard device and permitted test MQTT topic.
2. Define a JSON schema that contains stable device identity, timestamp and temperature.
3. Configure the EMQX rule to forward only the test topic; inspect the rule SQL, connector and sink delivery status.
4. Consume the Kafka test topic; confirm one expected record was written.
5. Test the bridge device mapping and ThingsBoard HTTP response.
6. Confirm ThingsBoard `Latest Telemetry` shows the test reading.
7. Add an alarm rule above a chosen threshold and test that one alarm is created.
8. Verify alarm clearing/debounce behavior and record a rollback.

Keep language and transport separate. A dashboard translation is not an SMS carrier. An SMS provider's delivery confirmation is not a Kafka ACK. This separation makes offline development easier to reason about.

Configure SMS providers and multilingual notifications

An offline development computer can simulate SMS events without Internet. Real SMS delivery requires access to a carrier, private SMPP connection or GSM modem; strict network isolation may prevent an external cloud SMS API from working. Choose the communications method deliberately.

Three approaches, from lowest risk to highest

Approach A: use existing ThingsBoard notification settings. The older Linux manual identified built-in Twilio, AWS SNS and SMPP support in the pinned ThingsBoard source. Check the installed CE version and account permissions; availability may vary by edition or configuration. Configure an approved provider without editing Java source, and test against authorized numbers only.

Approach B: use an external provider-neutral SMS adapter. Let ThingsBoard or the bridge send an authenticated internal alert to a separate service. The adapter selects a provider, renders a localized template, sends the message, records a delivery status and deduplicates retries. This keeps carrier code out of the ThingsBoard core.

Approach C: modify ThingsBoard's Java SMS sender implementation. This requires extending provider configuration, serialization, credential handling, factory dispatch, UI settings, tests and migration compatibility. Treat as a full application change; do not start here as a beginner.

Example localized alert contract

```
{
  "alarm_id": "alarm-2026-0001",
  "device_ref": "warehouse-1",
  "recipient_ref": "on-call-operations",
  "locale": "en",
  "template": "high_temperature",
  "parameters": {"temperature": 38.2, "unit": "C"}
}
```

A mock adapter can record the message and a synthetic result **without contacting a public SMS service**. Once the rest of the offline pipeline works, introduce an authorized real provider or local modem in a separate acceptance test. Avoid storing phone numbers in GitHub logs and screenshots.

Recommended alert quality checks

Verify proper language/placeholder formatting, authorized recipient lookup, one delivery per event + recipient + channel, retry on provider failure with bounds, cancellation after alarm clear, and audit log redaction. ThingsBoard device alarms, notification text and provider status are different records; design the correlation keys explicitly.

Windows/WSL2 route: use only if you choose the legacy distribution path

The original Source Kit PDF documented **Windows 10 + openSUSE Leap 16.0 WSL2 + Docker Desktop** as an alternative. It is **not** the same deployment topology as the VMware/openSUSE route in this document.

On an Internet-connected Windows host, stage verified installers and Windows features before you disconnect. The original release includes `WSL2-Installer.msi`, `Docker-Desktop-Installer.zip`, `openSUSE-Leap-16.0.wsl`, and `openSUSE-Leap-16.0.zip` assets. Those are convenience artifacts; validate signatures, OS compatibility and current support before using them.

Task	VMware / native Linux	Windows / WSL2
OS boot	openSUSE guest OS boots in VMware	openSUSE runs as a WSL2 distribution
Container engine	Docker Engine installed inside Linux	Docker Desktop on Windows with WSL integration
Storage	Prefer Linux filesystem inside virtual disk	Prefer WSL's Linux filesystem, not `/mnt/c` for heavy builds
Host dependencies	openSUSE RPM packages	Windows WSL prerequisites, Docker Desktop, WSL distro
Device passthrough	VMware USB/Bluetooth access must be configured	WSL USB/Bluetooth integration requires a separate plan
Network-isolation proof	Disconnect VM network adapter	Disconnect/deny all Windows + WSL + Docker routes deliberately

Do not attempt to install the WSL `.wsl` bundle into VMware as an ISO, and do not install the VMware Linux Docker Engine by following Docker Desktop Windows instructions. Use the matching version of the original Source Kit's WSL-specific manual if selecting this route.

Daily operations, service backups, and safe rollback

Start of each day

```
BASE="$HOME/projects/iot-offline-development"

# Read-only environment checks:
df -h /home
sudo docker info >/dev/null && echo DOCKER_OK
sudo docker compose -f "$BASE/compose.yaml" ps
sudo docker ps -a
```

If the services were deliberately stopped, use the existing, reviewed Compose file without rebuilding or pulling anything; never rerun initial database or KRaft setup on an already initialized environment.

```
# Existing initialized kit only:
# sudo docker compose -f "$BASE/compose.yaml" \
#   up -d --no-build --pull never
```

Before deploying a change

- Back up PostgreSQL and test a **restore to a separate empty volume**.
- Preserve Kafka KRaft metadata and data, plus topic/listener config.
- Preserve EMQX node data/cookies, secret credentials and rule/connector configuration.
- Preserve Zigbee2MQTT coordinator/device database before pairing/config changes.
- Record container images, JARs and source commits before swapping builds.
- Confirm a rollback path that **does not roll back the database schema blindly**.

What never belongs in public GitHub

`.env` files with passwords, database dumps, customer device data, private SMS keys, ThingsBoard access tokens, SSH private keys, EMQX cluster cookies and logs containing sensitive data. Upstream test fixture keys may appear in dependency trees; they are not inherently private, but require provenance and licensing review before public distribution.

Stop / rollback basics

`docker compose stop` stops containers without asking Docker to remove volumes. `docker compose down` removes containers/networks, which is more intrusive; **never add `-v` casually**. For rollback, restore the previously verified application image/artifact while preserving intended data. If a database migration occurred, use a separately tested downgrade or restore procedure compatible with the previous application version.

Troubleshooting: diagnosis before destructive changes

Symptom	Likely area	Safe first checks / next action
GitHub upload resets at 15%	Network/proxy/request size	Use the published 64 MiB chunks; rerun resumable uploader on staging computer.
`sha256sum: FAILED`	Incomplete/wrong download or storage error	Re-download bad asset and repeat checksum; inspect disk logs.
`ARCHIVE...PASS` but restore fails	Wrong folder, missing baseline, existing destination	Check `$HOME/projects/...`, script prerequisites, available space; do not overwrite.
Docker cannot connect	Docker service or permissions	`sudo systemctl status docker`; `sudo docker info`; check Compose v2.
Docker tries to pull an image	Missing local image or wrong tag	Inspect `docker images`; load original `.tar.gz` archive; use `--pull never`.
Maven offline dependency error	Wrong cache mount or missing artifact	Confirm `/offline-maven` contains repository layout; stage missing packages online.
Gradle offline cache miss	Gradle executable/plugin not cached	Check Gradle 9.2.1 path and `GRADLE_USER_HOME`; do not enable network.
EMQX Hex/Mix fetch requested	Full build needs uncached dependency	Record package and required version; module compilation is not full clean build.
Kafka startup storage error	KRaft metadata or listener mismatch	Inspect `meta.properties`, `server.properties`, `docker logs`; **do not format again**.
ThingsBoard blank/HTTP errors	DB initialization, Java backend or UI bundle	Inspect PostgreSQL health, ThingsBoard logs, port mapping, UI assets.
Bridge receives Kafka but no telemetry	Device token mapping or HTTP/API issue	Trace one test record, bridge logs (redacted), ThingsBoard Latest Telemetry.
No Bluetooth adapter	VM USB passthrough / host adapter absent	`lsusb`, `/sys/class/bluetooth`, `systemctl status bluetooth`; mock tests remain valid.
`Input/output error` in cache	Filesystem/disk corruption	Stop rewriting affected data; back up, inspect Btrfs/disk, restore affected asset.

Special recovery note: Btrfs checksum corruption on original VM

During release preparation, the original VM's `/dev/sda2` Btrfs filesystem reported checksum corruption in Maven's `protoc-3.25.5-linux-x86_64.exe`. The file was re-downloaded, verified against the upstream published checksum, and atomically replaced. The Maven archive then packaged and verified.

Do not interpret this as an all-clear for the storage device. The filesystem's `Data` profile was `single`, with no redundant data copy for automatic healing. Preserve irreplaceable sources/configuration on different media before a full scrub or further repairs. Do not run `btrfs check --repair` without expert recovery planning.

```
# Read-only, if using Btrfs:
sudo btrfs device stats /home
sudo btrfs filesystem usage -T /home
sudo journalctl -k -b --no-pager | grep -i 'btrfs' | tail -50
```

Errors growing during reads/writes are a reason to pause deployment and verify the host disk/VM image. A successful gzip/tar integrity test proves only that the tested archive stream was readable at that moment.

GitHub release publishing and license review

The published Linux release was completed only after the large first-asset upload failed and the seven tar archives were segmented into **82 fixed 64 MiB upload chunks**. The upload checker confirmed **88 required new assets**, and a separate GitHub release check showed **90 total uploaded assets**, approximately 5.23 GB.

Verify publication without downloading again

Open the Linux release page on a connected machine. Check for these **six support files**:

```
SHA256SUMS-CHUNKS
SHA256SUMS-PORTABLE
SOURCE-REVISIONS.txt
INSTALL.txt
restore-portable-environment.sh
assemble-portable-chunks.sh
```

Then check that 82 `.chunk-*` assets are present. Reassembly checks both chunk hashes and reconstructed full-archive hashes. **These checks do not prove the assets are safe for production or licensed for any commercial use.**

Legal and secret-scanning gates

- Verify upstream license and third-party dependency notices before redistributing archives publicly. In particular, EMQX Enterprise 6.3.1 is **BSL 1.1**, not an unrestricted Apache-licensed enterprise product.
- Scan actual archive contents and, where applicable, Git object history for credentials; filename-based checks can miss tokens embedded in code or configuration.
- Distinguish upstream test fixture keys from private generated credentials. In the release preparation, two locally modified test keys in `greptimedb_rs` were replaced with original upstream content in a release-only snapshot.
- Keep the published version manifest, SHA-256 manifests, restore scripts and limitations in the release description.
- Publish release assets separately from ordinary source Git history. Do not push multi-gigabyte cache trees directly into `main`.

Publishing changes on the offline laptop: collect small source patches and logs locally. Move reviewed patches to the connected staging computer for GitHub upload. Never give a production VM broad Internet access merely to push a commit.

Full offline acceptance test: prove the new laptop really works

The key remaining milestone is an **independent second-machine test**. Use a fresh VM or new laptop with no preexisting `iot-offline-development` and `iot-offline-build-sandbox` directories. Take the release assets from external storage, unplug/disconnect the network, then run this checklist in order.

No.	Acceptance gate	Required evidence
01	OS/architecture correct	`/etc/os-release`, `uname -m`, disk space report.
02	Host prerequisites available offline	`docker info`, `docker compose version`, `git --version`, Python, tar and checksum utility.
03	Source Kit verified	`COMPLETE OFFLINE PACKAGE VERIFICATION: PASS`.
04	Baseline restored to new destination	Source SHAs correct, caches/runtime present, restore reports PASS.
05	DevKit chunks verified	82 chunk names; `sha256sum -c SHA256SUMS-CHUNKS` all OK.
06	DevKit archives reassembled	`PORTABLE_ARCHIVES_REASSEMBLED=PASS`.
07	DevKit sandbox restored	`PORTABLE_OFFLINE_ENVIRONMENT_RESTORED=PASS`.
08	Docker builder images available	Local `docker image inspect` for each required pinned image.
09	Kafka modified-source build	`--offline` build finishes, changed-source compile evidence recorded.
10	ThingsBoard backend modified-source compile	Maven `-o` build reports success with correct cache mount.
11	ThingsBoard UI and Zigbee builds	Offline frontend and pnpm build pass, without network.
12	EMQX targeted module compile	Targeted compile passes; whole-enterprise clean build remains separate gate.
13	Runtime data/database prepared	Backup or fresh database installation independently validated; no overwritten data.
14	Functional message path	Test MQTT → EMQX → Kafka → bridge → ThingsBoard Latest Telemetry.

No.	Acceptance gate	Required evidence
15	Restart, persistence, rollback	Reboot VM, check retained state, restore baseline artifact with no data loss.
16	Hardware and production gates	BLE, Zigbee, SMS provider, secrets and licensing tested separately.

Pass criteria: all mandatory gates for your intended use must be supported by dated logs and screenshots. A successful `sha256sum` does not replace a real modified-source build, and a successful offline build does not replace live hardware or operational resilience testing.

Suggested evidence record

```
Machine ID / OS: _____________________________
Offline method: VM NIC disconnected / other: ___
Source Kit tag: v0.1.0-offline-preview
DevKit tag: v0.1.0-offline-linux-dev-preview
Baseline restore: PASS / FAIL / NOT RUN
Chunk verify + reassembly: PASS / FAIL / NOT RUN
Sandbox restore: PASS / FAIL / NOT RUN
EMQX module compile: PASS / FAIL / NOT RUN
Kafka jar compile: PASS / FAIL / NOT RUN
ThingsBoard Java compile: PASS / FAIL / NOT RUN
ThingsBoard UI bundle: PASS / FAIL / NOT RUN
Zigbee2MQTT build: PASS / FAIL / NOT RUN
Java bridge compile: PASS / FAIL / NOT RUN
MQTT-to-ThingsBoard integration: PASS / FAIL / NOT RUN
Persistence/reboot: PASS / FAIL / NOT RUN
Reviewed by / date: _____________________________
```

Quick command reference for new maintainers

Commands below assume a **new, empty target with baseline and assets already staged**. Read the corresponding chapter before performing state-changing commands.

Purpose	Command summary
Check Linux	`cat /etc/os-release; uname -m; df -h /home`
Check Docker	`sudo docker info; sudo docker compose version`
Verify baseline	`bash scripts/restore-complete-offline-kit.sh --verify` from Source Kit root
Restore baseline	`bash scripts/restore-complete-offline-kit.sh --restore "$HOME/projects/iot-offline-development"`
Verify chunks	`sha256sum -c SHA256SUMS-CHUNKS` in flat chunk directory
Assemble archives	`bash assemble-portable-chunks.sh`
Restore sandbox	`bash restore-portable-environment.sh` after baseline exists
Verify image archives	`bash scripts/restore-offline-docker-images.sh --verify` from Source Kit root
Load images	`bash scripts/restore-offline-docker-images.sh --load`
Verify runtime preparation	`bash scripts/prepare-restored-offline-kit.sh --verify`
Prepare fresh runtime	`bash scripts/prepare-restored-offline-kit.sh --prepare "$BASE"`
Verify database dump	`bash scripts/restore-offline-postgres.sh --verify-backup "$PRIVATE_DUMP"`
Verify start readiness	`bash scripts/start-offline-services.sh --verify "$BASE"`
Start fresh restored kit	`bash scripts/start-offline-services.sh --start "$BASE"`
Show container status	`sudo docker ps -a`
Show recent logs	`sudo docker logs --tail=80 iot-thingsboard`
Check Git state	`git -C "$DEV/isolated-sources/kafka" status --short`

No destructive shortcuts: do not delete existing destination directories merely to make an installer proceed. Inspect why a prerequisite failed, preserve working state, and use a new disposable VM if necessary.

Glossary, version pins, and source references

Plain-language glossary

Term	Meaning
Archive / release asset	A file downloaded from a tagged GitHub release rather than source control.
Chunk	One small transport segment of a larger archive; reassemble before extraction.
SHA-256	A cryptographic fingerprint used to detect changed/corrupt bytes.
Dependency cache	Local copies of libraries that a compiler would otherwise download.
Builder image	Docker image containing compilers/tools; different from a running application image.
Runtime	The actual services, application files, databases and configuration.
Offline build	Compiles without fetching dependencies from the Internet.
Clean build	Recompilation from a clean source/build state, without relying on already compiled outputs.
KRaft	Kafka metadata/storage mode; its cluster identity must not be reformatted casually.
MQTT / QoS 1	Lightweight message protocol / at-least-once delivery acknowledgment.
SMPP	A telecommunications protocol for SMS messaging.
Btrfs	Linux filesystem supporting checksums and snapshots; data corruption can still occur.
Provenance	Whether a file matches its expected source, commit or upstream package.

Pinned Git source revisions

```
EMQX Enterprise 6.3.1:
 b0604926195f9372b0068b61f88d409d639a2401
Apache Kafka 4.2.2:
 c5e942859576203e1b00260070fe6241b64db251
ThingsBoard CE 4.3.1.6:
 c77335f4a524bb9fe0209806672c6ec7aea76141
Zigbee2MQTT 2.14.2:
 344b6e03b43644c97154a72ca9eae17193c553a2
```

These revisions are from the version manifest in the verified development kit preparation. Consult the release's `SOURCE-REVISIONS.txt` file on installation day rather than assuming a later reissued release kept the same pins.

Source documents and release references

This edition was built from the two earlier manuals and the corresponding published repository scripts/status documents; it also incorporates the verified release upload and integrity checks completed on 13 November 2026.

1. **Source Kit release:** https://github.com/Developer20003/iot-offline-source-kit/releases/tag/v0.1.0-offline-preview
2. **Linux DevKit release:** https://github.com/Developer20003/iot-offline-linux-devkit/releases/tag/v0.1.0-offline-linux-dev-preview
3. **Source Kit repository:** https://github.com/Developer20003/iot-offline-source-kit
4. **Linux DevKit repository:** https://github.com/Developer20003/iot-offline-linux-devkit
5. **Linux build verification matrix:** https://github.com/Developer20003/iot-offline-linux-devkit/blob/main/docs/OFFLINE-BUILD-MATRIX.md
6. **Linux command guide:** https://github.com/Developer20003/iot-offline-linux-devkit/blob/main/docs/COMMAND-GUIDE.md
7. **Linux customization workflow:** https://github.com/Developer20003/iot-offline-linux-devkit/blob/main/docs/CUSTOMIZATION-WORKFLOW.md
8. **EMQX licensing in pinned source:** https://github.com/emqx/emqx/blob/b0604926195f9372b0068b61f88d409d639a2401/LICENSE

Previous editions incorporated: Offline IoT Development Kit — Complete Installation, Build, Editing, Customization and Troubleshooting Handbook (30 October 2026, 33 pages) and Offline Linux IoT Platform — Beginner's Customization & Development Guide (10 November 2026, 27 pages). This edition supersedes their status summaries for the newly published companion assets; it does **not** claim a fresh-machine certification that was never performed.

Final installation and handover checklist

- ☐ Correct OS, disk space and Docker/Compose prerequisites verified
- ☐ Source Kit release assets and original repository acquired
- ☐ Linux DevKit 90-asset snapshot acquired and transferred
- ☐ Both checksum manifests verified before extraction
- ☐ Original baseline restored to a new, nonexisting target
- ☐ All 82 data chunks reassembled into seven portable archives
- ☐ Portable sandbox restored without overwriting existing data
- ☐ Runtime database restoration / new-schema plan validated
- ☐ Builder images present locally with no Internet pull
- ☐ Changed-source offline compilation and logs collected
- ☐ MQTT → Kafka → bridge → ThingsBoard verified on target
- ☐ Restart/persistence/rollback tested and documented
- ☐ Physical BLE/Zigbee and external SMS limitations understood
- ☐ Secrets, licensing and distribution terms reviewed

Handover standard: The installation is finished only when a second person can follow this manual on a fresh compatible machine, reproduce the critical checks with networking disabled, explain every unresolved test and restore the previous working state without loss of data.